420-KBC-LG : DÉVELOPPEMENT D'APPLICATION MOBILE-2

Leçon 9 : Navigation et Expo Router

Lina Jabbour

Hiver 2026

Sujets

A. Navigation
B. Expo-router
C. Créer un projet avec expo router
D. Stack et Tabs
E. Routes dynamiques et passage d'info

NAVIGATION

Dans une application, la **navigation** c'est

- Se repérer entre les différents écrans
- Trouver comment aller vers un autre écran

C'est la gestion de la présentation et de la transition entre plusieurs écrans.

Expo Router vs React Navigation

- **React Navigation** est une bibliothèque qui permet d'implémenter des fonctionnalités de navigation dans une application React Native. C'est le standard historique.

- **Expo Router** est la nouvelle façon de faire de la navigation avec Expo. Expo-router est batit sur React Navigation et se sert du système de fichiers. Il est plus simple et similaire au web.

Expo Router vs React Navigation

En bref :

- **Expo Router** est idéal pour les débutants, les projets Expo, une navigation rapide, et applications Web/Mobile (Universal Apps).
- **React Navigation** est préférable pour les projets personnalisés, les structures très complexes, ou hors de l'écosystème *expo*.

B-Les concepts de base **d'expo router**

1. Toutes les pages sont dans le dossier **app,** qui sert exclusivement à définir les routes de votre application.
2. Vous avez besoin des fichiers
   a. **index** dans le dossier, la page qui ouvre par défaut.
   b. **_layout.tsx (.js)** pour configurer la navigation à ce niveau.
3. Vous aurez la plupart du temps ces 2 fichiers dans chaque sous dossier de **app**.

```
/app                 root
  index.tsx          screen
  _layout.tsx        layout file
package.json
```

Et le _layout à la racine, directement sous **app**, est toujours un **Stack** navigator.

https://docs.expo.dev/router/basics/core-concepts/

```
app
  _layout.js        U
  index.js          U
  page2.js          U
> app0 - Copie
> assets
> node_modules
  .gitignore
  app.json          2, M
  index.js
  package-lock.json M
  package.json      M
```

```
4
5   import { Stack } from 'expo-router'
6
7   const RootLayout = () => {
8       return (
9           <Stack>
10              <Stack.Screen
11                  name="index"
12                  options={{ headerShown: false }} />
13          </Stack>
14      )
15  }
16
17  export default RootLayout
```

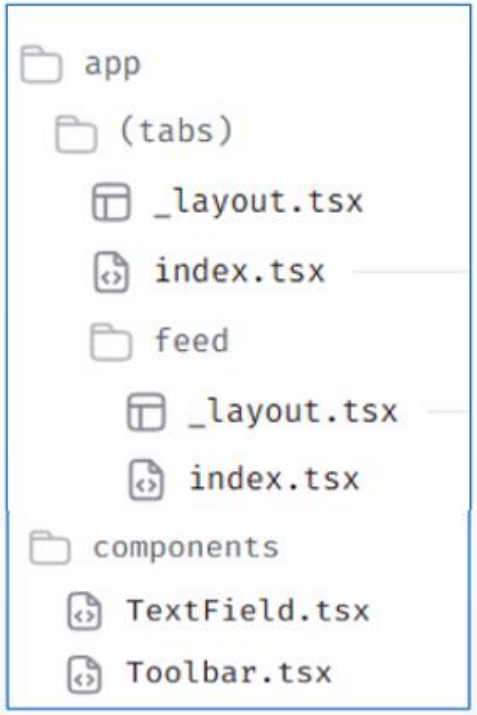
Les concepts de base **d'expo router**

4. **_layout.tsx (.js)** configure l'ordre de la navigation Stack ou Tab dans dossier.

5. (tabs) un sous-repertoire qui sert à afficher les tabs en bas de l'ecran; entre parenthèses pour ne pas avoir à specifier *tabs* dans l'adresse

6. (tabs)/index: écran par défaut/page principale/accueil

7. Les autres composants doivent être dans un répertoire de niveau supérieur, comme **components** dans cet exemple.



- https://docs.expo.dev/router/basics/core-concepts/

Navigation Tab

Consider the following navigation tree:

```
app
  (tabs)
    _layout.tsx
    index.tsx                                   single page tab
    feed
      _layout.tsx                               tab with a stack inside
      index.tsx
      [postId].tsx
    settings.tsx                                single page tab
```

In the **app/(tabs)_layout.tsx** file, return a `Tabs` component:

app/(tabs)/_layout.tsx Copy

```
import { Tabs } from 'expo-router';

export default function TabLayout() {
  return (
    <Tabs screenOptions={{ headerShown: false }}>
      <Tab.Screen name="index" options={{ title: 'Home' }} />
      <Tab.Screen name="feed" options={{ title: 'Feed' }} />
      <Tab.Screen name="settings" options={{ title: 'Settings' }} />
    </Tabs>
  );
}
```

In the **app/(tabs)/feed/_layout.tsx** file, return a `Stack` component:

https://docs.expo.dev/router/basics/common-navigation-patterns/

Tab: Dans _layout.tsx/js: 3 liens pour 3 pages en **bas** de l'écran avec des icônes

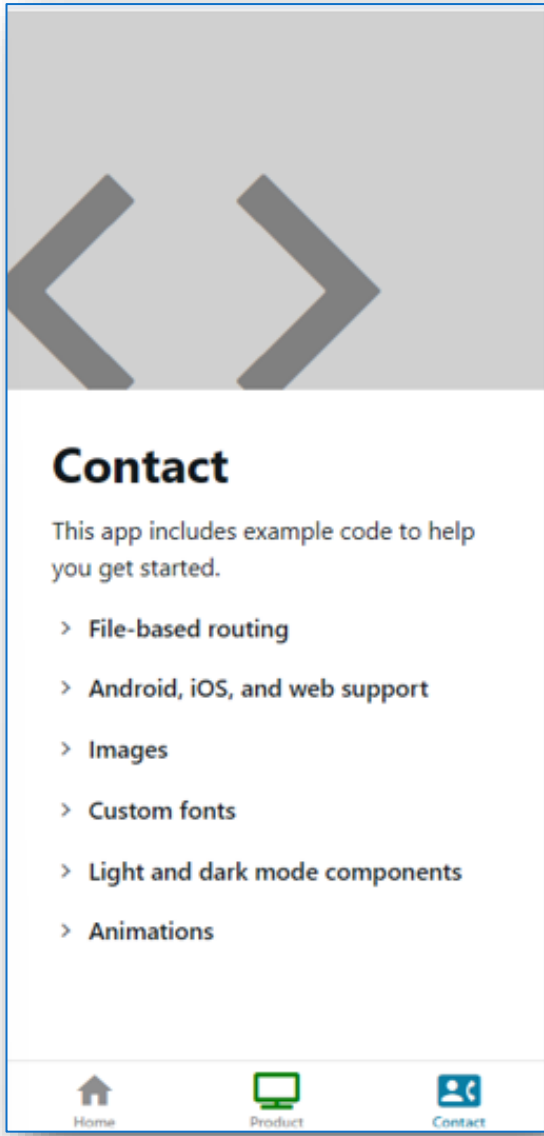
NAV_ER
- .expo
- .vscode
- app
  - (tabs)
    - _layout.tsx M
    - contact.tsx U
    - index.tsx M
    - produits.tsx U
  - _layout.tsx
  - +not-found.tsx
- assets
- components
  - __tests__
    - __snapshots__
      - ThemedText-test....
    - ThemedText-test.tsx
  - ui
    - IconSymbol.ios.tsx
    - IconSymbol.tsx 1
    - TabBarBackground....
    - TabBarBackground....
  - Collapsible.tsx
  - ExternalLink.tsx
  - HapticTab.tsx
  - HelloWave.tsx
  - ParallaxScrollView.tsx

_layout.tsx 1, M | produits.tsx U | IconSymbol.tsx 1, M | IconSymbol.ios.tsx | app.js

app > (tabs) > _layout.tsx > TabLayout

```
12  export default function TabLayout() {
22          tabBarStyle: Platform.select({
27            default: {},
28          }),
29        }}>
30        <Tabs.Screen
31          name="index"
32          options={{
33            title: 'Home',
34            tabBarIcon: ({ color }) => <IconSymbol size={28} name="house.fill" color={color} />,
35          }}
36        />
37        <Tabs.Screen
38          name="produits"
39          options={{
40            title: 'Product',
41            tabBarIcon: ({ color }) =><FontAwesome5 name="tv" size={24} color="green" />
42
43          }}
44        />
45        <Tabs.Screen
46          name="contact"
47          options={{
48            title: 'Contact',
49            tabBarIcon: ({ color }) => <IconSymbol size={28} name="contact" color={color} />,
50          }}
51        />
52      </Tabs>
53    );
54  }
```

name contient le nom du ficher sans l'extension



C- Créer un projet expo router

On peut créer un projet **expo router** en utilisant entre autre une des suivants:

1-npx create-expo-app@latest nomProjet :

Cré un exemple de app avec expo-router et un peu trop de code.

2- npx create-expo-app@latest --template **tabs** nomProjet

Installe et configure des fichiers avec Expo Router et TypeScript activés

3-npx create-expo-app@latest --template **blank** nomProjet

et après on installe les bilbliothèques dont nous avons besoin.

nom du projet : lettres (a-z, A-Z), chiffres (0-9) et trait d'union (-), *exemple: nav-blank*

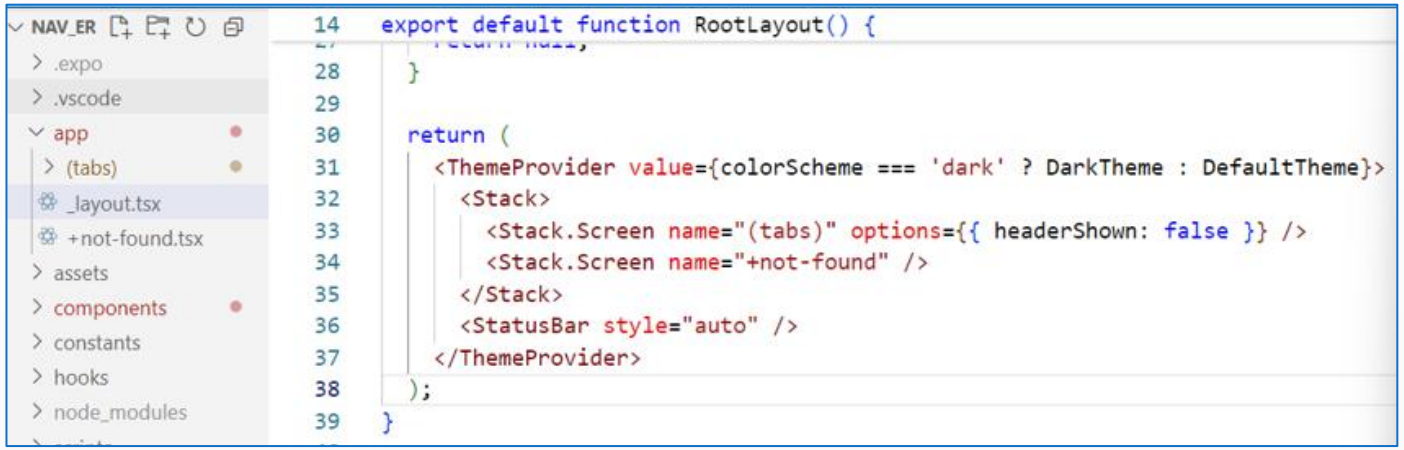
https://docs.expo.dev/more/create-expo/

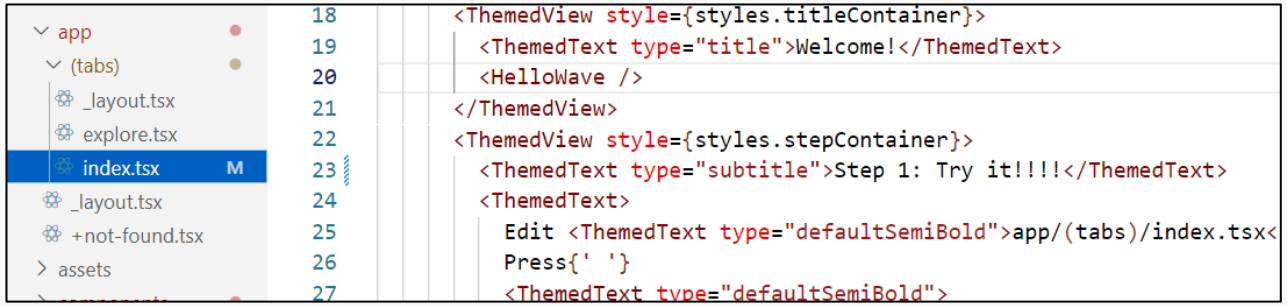
1-Expo-router

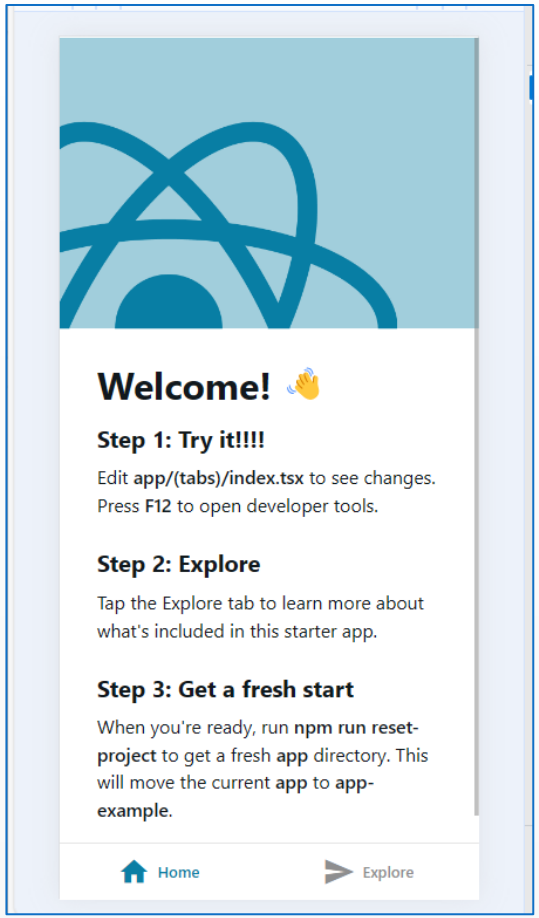
npx create-expo-app@latest nomProjet

Cré un exemple de app avec expo-router et un peu trop de code.

https://docs.expo.dev/router/introduction/



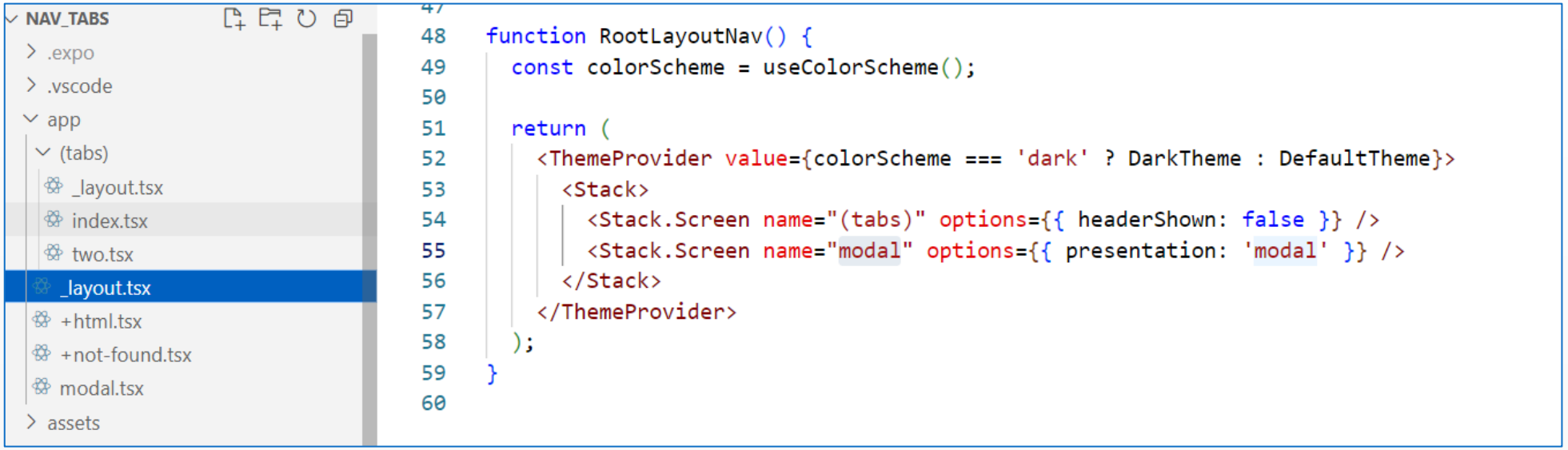




2-create-expo-app avec –template **tabs**

>npx create-expo-app@latest –template tabs nomProjet

tabs: Installe et configure des fichiers avec expo Router et TypeScript activés.



https://docs.expo.dev/more/create-expo/

C-Créer un projet expo react avec template blank

à faire en classe avec le prof ☺

Créer un projet expo react blank (comme d'habitude :)

1. Créez un projet: npx create-expo-app@latest --template blank **nav-blank**
2. Dans le dossier du projet, faites les installations >npx expo install expo-router react-native-safe-area-context react-native-screens expo-linking expo-constants expo-status-bar

3. Configurez le point d'entrée dans package.json:

"main": "expo-router/entry"

```
{} package.json M ×
{} package.json > {} dependencies
1  {
2    "name": "nav-blank",
3    "version": "1.0.0",
4    "main": "expo-router/entry",
     ▷Debug
5    "scripts": {
6      "start": "expo start",
```

4. Dans app.json ajoutez:

"scheme": "nav-blank"

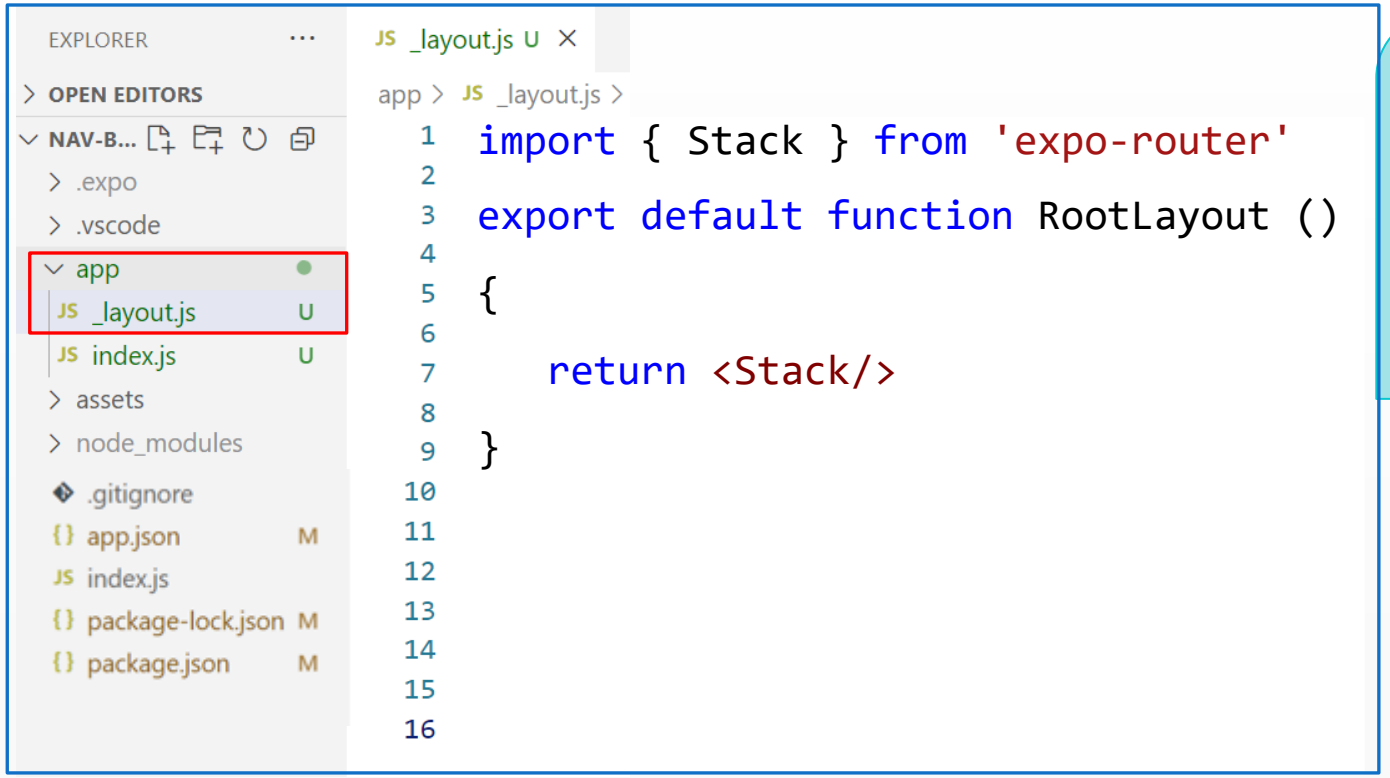
```
{} app.json M ×
{} app.json > {} expo
1  {
2    "expo": {
3      "name": "nav-blank",
4      "slug": "nav-blank",
5      "scheme": "nav-blank",
6      "version": "1.0.0",
```

5. Supprimez App.js (on n'en a plus besoin)
6. Exécutez... cela ne fonctionnera pas ... pas encore ☺

- https://medium.com/@jwbrendan/an-implementation-of-expo-router-with-typescript-tab-navigators-and-stack-navigators-in-a-react-f0655d12d86b

Implémentez la navigation

Créez le dossier **app** et les fichiers _layout.js et index.js Code du _layout.js:



Il se peut que app et index soient créés automatiquement à la premières exécution après la configuration pour expo router

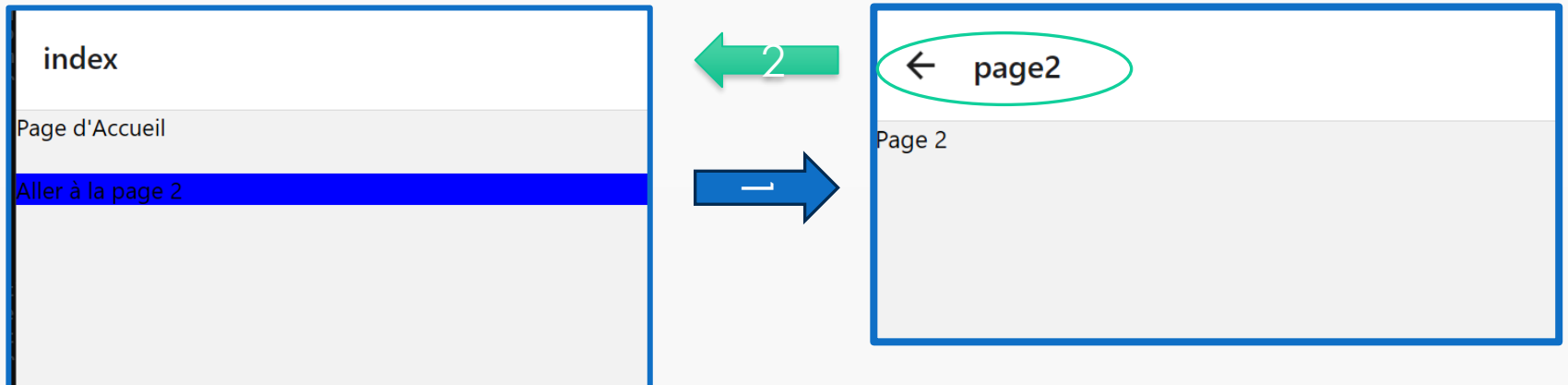
Aller à une autre page **Link**

app > index.js > default

```
// app/index.js
import { View, Text } from "react-native"
import { Link} from 'expo-router'

const HomePage = () => {
    return (
        <View >
            <Text>Page d'Accueil</Text>
            <Text> </Text>
            <Link style={{backgroundColor: "blue"}}
               href="/page2">Aller à la page 2</Link>
        </View>
    )
}
export default HomePage
```

app > page2.js > default

```
// app/page2.js
import { View, Text } from "react-native"


const Page2 = () => {
    return (
        <View >
            <Text>Page 2</Text>

        </View>
    )
}

export default Page2
```



Aller à une autre page **Link**

Par défaut, Link ne peut contenir que du texte. Vous pouvez utiliser Pressable ou d'autres composants plus sophistiqués; pour activer la propriété onPress à l'intérieur d'un Link ajoutez la propriété **asChild**. **push** est utilisé pour assurer un ajout à la pile

```
import { Link} from 'expo-router'

const HomePage = () => {
    return (
        <View >
            <Text>Page d'Accueil</Text>
            <Text> </Text>
            <Link href="/page2" push>Aller à la page 2</Link>

            <Link href="/page2" asChild>
                <Pressable>
                    <Text>Page2 -Pressable</Text>
                </Pressable>
            </Link>
        </View>
    )
}
export default HomePage
```

Aller à une autre page avec **useRouter**

useRouter est le hook utilisé pour naviger

router.navigate: ajoutera une nouvelle page sur la pile,
ou achemine à une route existante sur la pile.

router.push pour pousser explicitement une nouvelle page sur la pile, (comme **Link**)

router.back pour revenir à la page précédente

router.replace pour remplacer la page actuelle sur la pile.

```
import { useRouter } from 'expo-router’;

export default function Home() {
  const router = useRouter();

  return <Button title="Aller à la page 2 BUTTON"
        onPress= {()=>router.navigate('/page2')} />

}
```

https://docs.expo.dev/router/basics/navigation/

Navigation entre les pages

Avec Expo Router, vous faites référence aux pages via leur URL et leur position relatif au répertoire app

```
<Button title="go to about page" onPress= {()=>router.navigate('/about')} />
```

```
app
  index.tsx ———————————————— router.navigate("/")
  about.tsx ———————————————— router.navigate("/about")
  profile
    index.tsx ———————————— router.navigate("/profile")
    friends.tsx ———————— router.navigate("/profile/friends")
```

https://docs.expo.dev/router/basics/navigation/

routes relatives

URL relative: on peut aussi naviguer par rapport à la route courante, commençant par

- **/** racine
- **./** (pour le répertoire courant)
- ou **../** (pour le répertoire parent)

✓ `<Link href="./article">Aller à l'article</Link>`

✓ `router.navigate('./article');`

Notez que **souvent il vaut mieux utiliser le chemin absolue:**

`href="/produits/article" quand le chemin est app/produits/article`

Exemple 2: Dossier (tabs):_layout.js et index.js

Dans app, créez le dossier (tabs) et les fichiers **_layout.js** et **index.js** avec le code:

```
app
| _layout
| (tabs)
|    | _layout
|    | index
```

```
// app/(tabs)/index.js
import { View, Text, Pressable } from "react-native"
import { Link, router } from 'expo-router'
const HomePage = () => {
  return (<View>
    <Text>Page d'Accueil</Text>
    <Link href="/tab_1"> Aller au tab 1</Link>
    <Pressable style={{backgroundColor:"green"}}
               onPress={() => router.push('/tab_2')}>
        <Text>Aller au tab 2</Text>
    </Pressable>
   </View>)
}
export default HomePage
```

```
// app/(tabs)/_layout.js
import { Tabs } from 'expo-router'
const TabsLayout = () => {
    return (<Tabs>
       <Tabs.Screen name="index" options={{headerTitle: "Accueil ", title: "Tab Accueil"}} />
       <Tabs.Screen name="tab_1/index" options={{headerTitle: "Tab 1", title: "Tab 1" }}  />
       <Tabs.Screen name="tab_2/index" options={{ headerTitle: "Tab 2", title: "Tab 2"}}/>
   </Tabs>)
}
export default TabsLayout
```

La structure de vos dossiers et fichiers index:js

Dans (tabs), créez les dossiers tab_1 et tab_2, fichiers **index.js** avec le code:

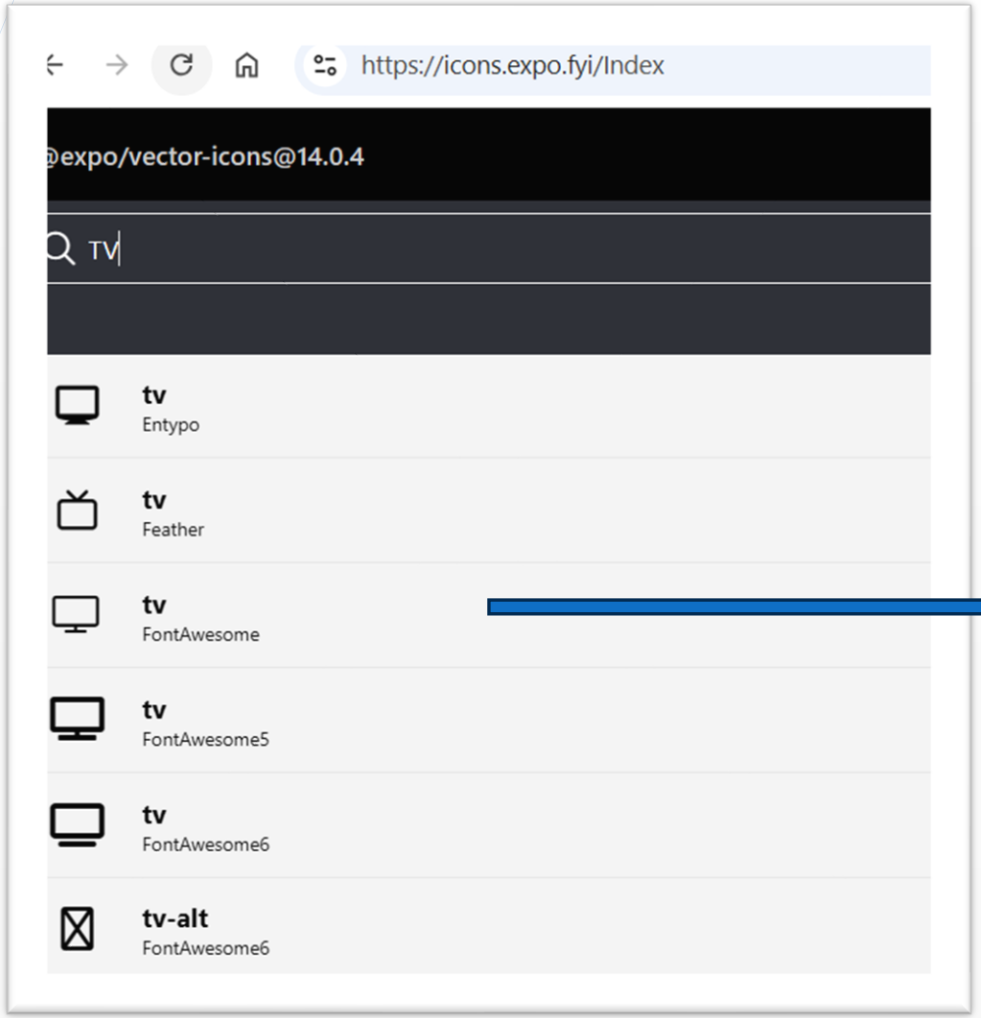
```
app
| _layout
| (tabs)
|    | _layout
|    | index
|    | tab_1
|    |    | index
|    | tab_2
|         | index
```

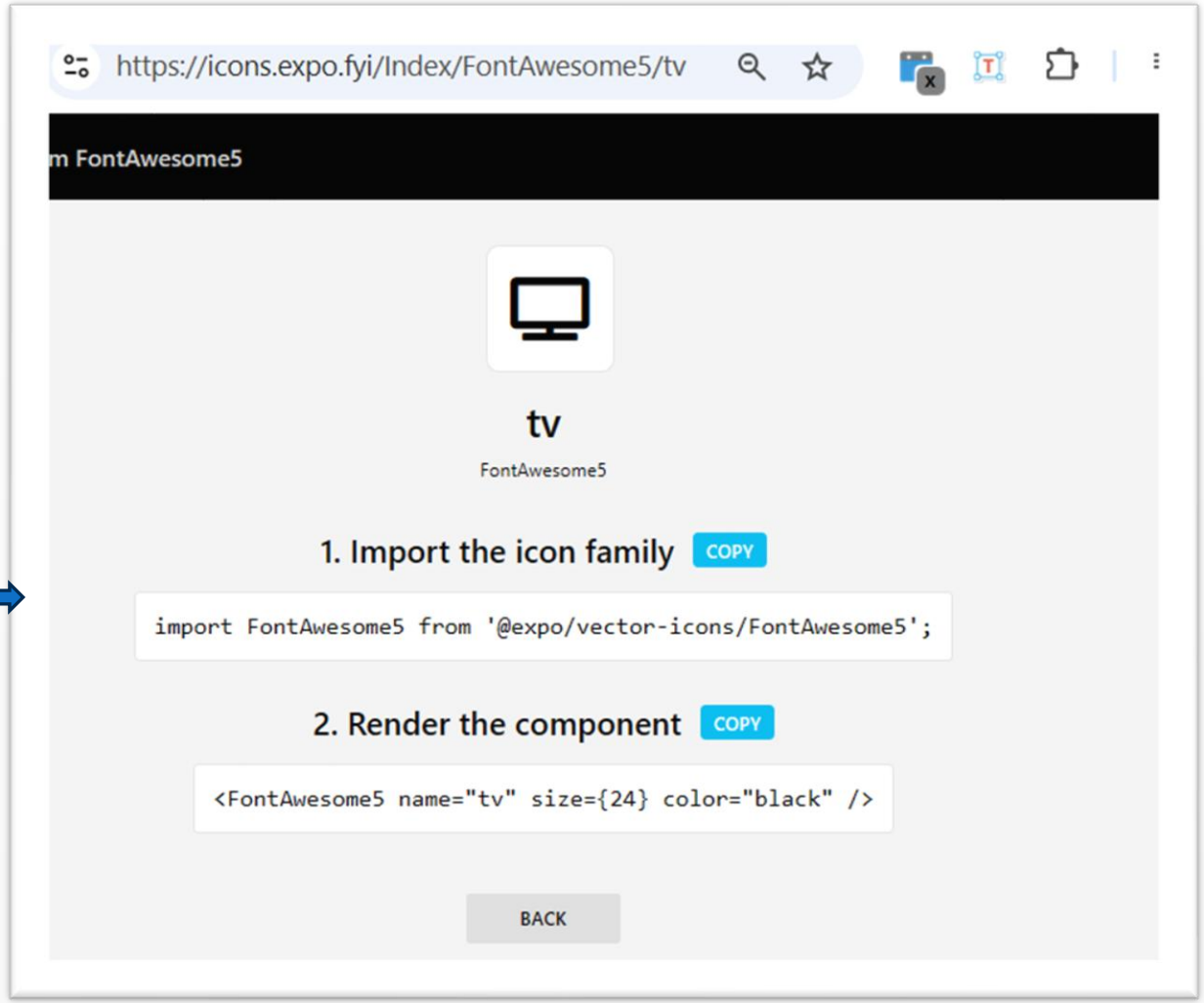
```
// app/(tabs)/tab_1/index.js
import { Text } from 'react-native'

const index = () => {
    return (
        <Text>Tab 1: Autobus</Text>
    )
}
export default index
oooooooooooooooooooooooooooooooo

// app/(tabs)/tab_2/index.js
import { Text } from 'react-native'

const index = () => {
    return (
        <Text>Tab 2: Avion</Text>
    )
}
export default index
```

7:06
Accueil
Page d'Accueil
Aller au tab 1
Aller au tab 2
Tab Accueil
Tab t1
Tab t2

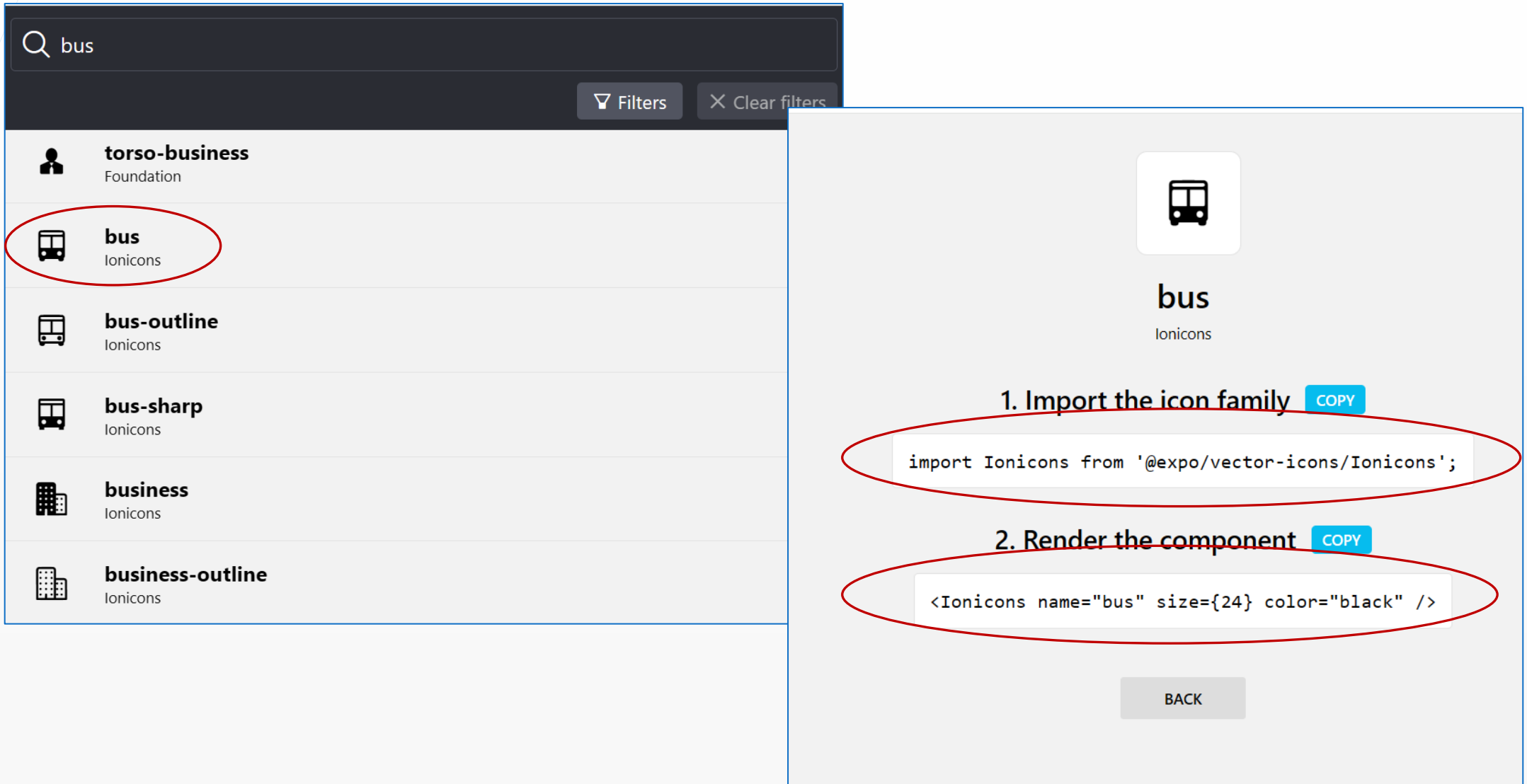
7:06
Tab 1
Tab 1: Autobus
Tab Accueil
Tab t1
Tab t2

Les icônes: https://icons.expo.fyi/Index

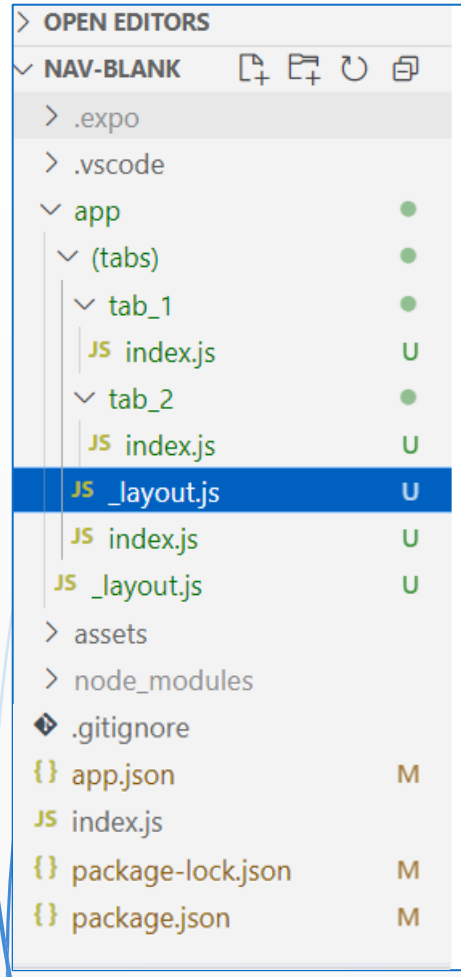




Les icônes: https://icons.expo.fyi/Index



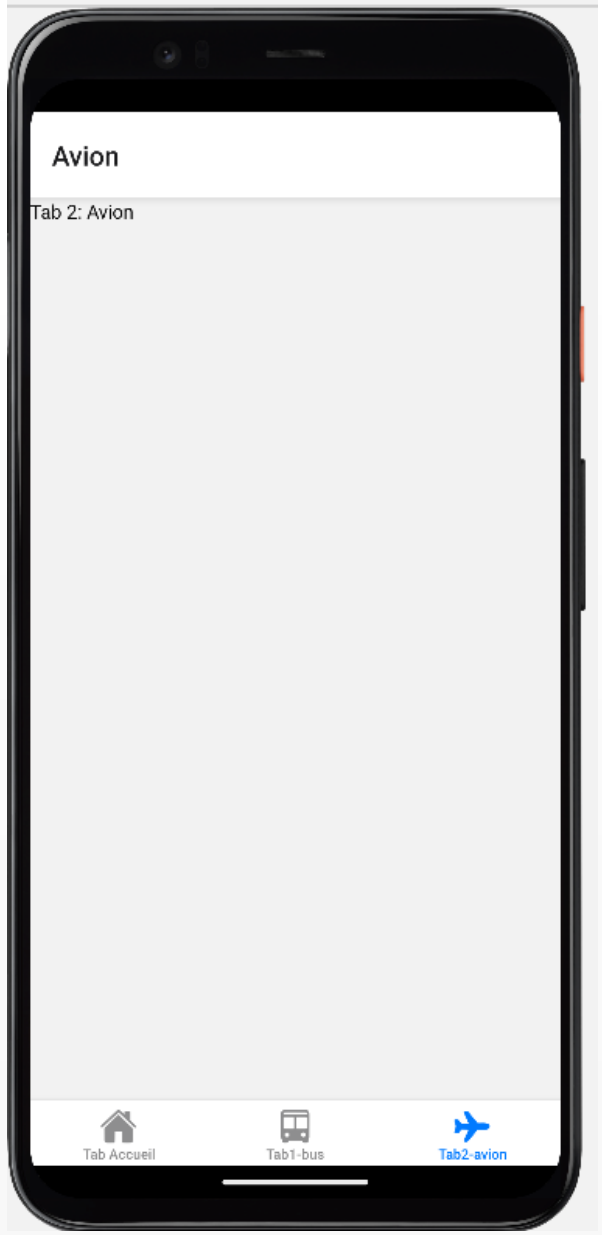
options: tabBarIcon, pour l'icône



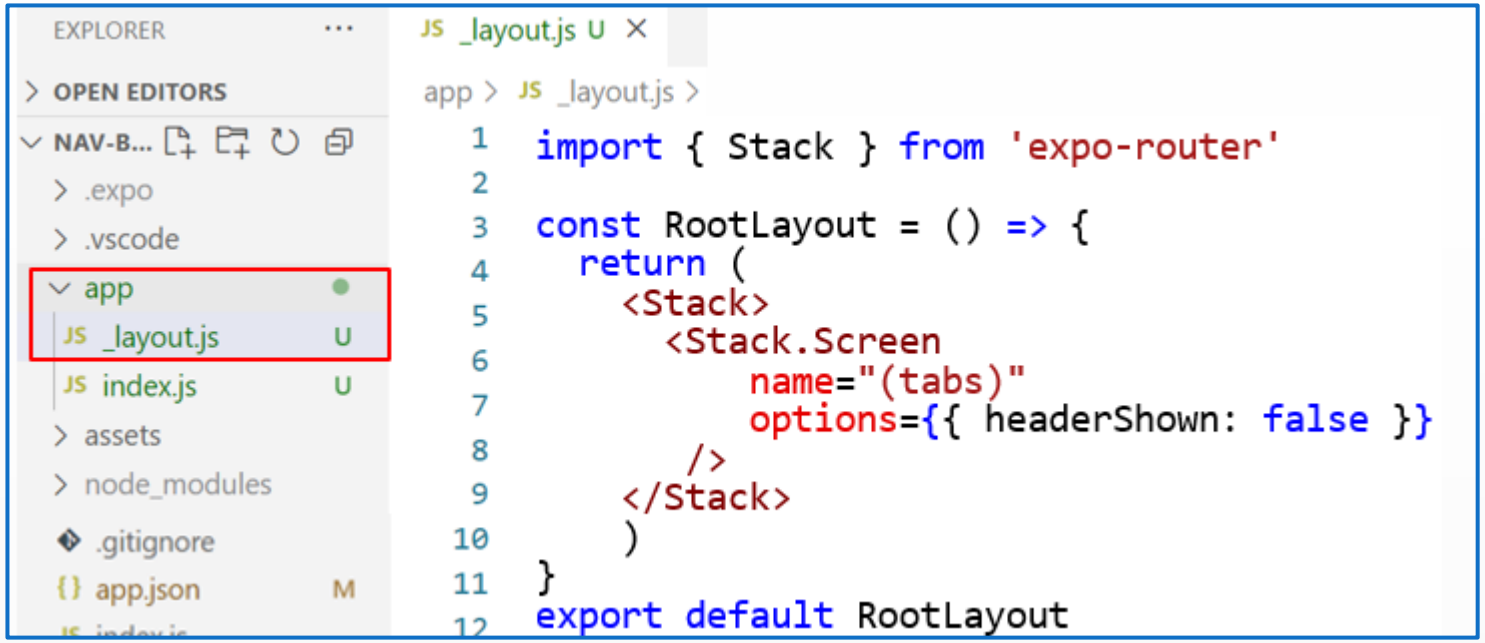
app > (tabs) > _layout.js > default

```
1   // app/(tabs)/_layout.js
2   import { Tabs } from 'expo-router'
3   import Ionicons from'@expo/vector-icons/Ionicons'
4
5   const TabsLayout = () => {
6       return (
7           <Tabs>
8             <Tabs.Screen name="index" options={{headerTitle: "Accueil ", title: "Tab Accueil",
9              tabBarIcon: ({color}) => <Ionicons size={28} name="home" color={color} /> }}
10             />
11              <Tabs.Screen name="tab_1/index" options={{headerTitle: "Autobus",  title: "Tab1-bus",
12               tabBarIcon: ({color}) => <Ionicons size={28} name="bus" color={color} /> }}
13              />
14              <Tabs.Screen name="tab_2/index" options={{ headerTitle: "Avion", title: "Tab2-avion",
15               tabBarIcon: ({color}) => <Ionicons size={28} name="airplane" color={color} /> }}
16               />
17          </Tabs>
18      )
19  }
20
21  export default TabsLayout
```

Après quelques modifications



Stack exemple 2



Reload avec expo router

Il vaut mieux **redémarrer expo** sur votre cellulaire/emulateur et refaire **npm start** quand vous modifiez la navigation avec expo router pour être sur que les modifications ont été appliquées

D-expo router: Stack et Tabs

- **Stack**: navigation en pile; où la navigation vers un nouvel itinéraire pousse un écran sur une pile, et le retrait de cet itinéraire le fait sortir de la pile.
- **Tabs**: navigation entre les écrans, en bas de l'écran.

```
import { Stack } from 'expo-router'

const StackLayout = () => {
  return <Stack>
    <Stack initialRouteName="busIntro" />
  </Stack>
}

export default StackLayout
```

```
import { Tabs } from 'expo-router'

const TabsLayout = () => {
 return (
  <Tabs>
    <Tabs.Screen name="index" options={{headerTitle: "Accueil ", title: "accueil"}} />
    <Tabs.Screen name="bus" options={{headerTitle: "Autobus",  title: "bus"}} />
    <Tabs.Screen name="avion/index" options={{ headerTitle: "Avion", title: "avion" }} />
  </Tabs>)
}

export default TabsLayout
```

Tabs.Screen, options

```
<Tabs>
    <Tabs.Screen name="tab_2/index"
        options={{headerTitle: "Avion", title: "avion",
            tabBarIcon:({color}) => <Ionicons name="airplane"
                                size={28} color={color} /> }}/>
    ....
</Tabs>
```

name: nom du fichier, et nom du tab si title est absent.

options:

- **headerTitle** : l'entête de la page, si non caché
- **title**: titre sous le icône
- **tabBarIcon**:fonction qui renvoie un composant (généralement une icône) à afficher.

```
(property) tabBarIcon?: ((props: {
    focused: boolean;
    color: string;
    size: number;
}) => React.ReactNode) | undefined
```

Tabs options

```
return (
    <Tabs screenOptions={{tabBarActiveTintColor: "pink"}}>
      <Tabs.Screen name="index" options={{headerTitle: "Accueil ", title: "accueil",
         tabBarIcon: ({color}) => <Ionicons size={28} name="home" color={color} /> }}
      />
       <Tabs.Screen name="bus" options={{headerTitle: "Autobus",  title: "bus",
          tabBarIcon: ({color}) => <Ionicons size={28} name="bus" color={color} /> }}
       />
       <Tabs.Screen name="avion/index" options={{ headerTitle: "Avion", title: "avion",
          tabBarIcon: ({color}) => <Ionicons size={28} name="airplane" color={color} /> }}
        />
    </Tabs>
)
```

screenOptions va effecter tous les Tabs

E- Passage d'information

index.js U

app > (tabs) > index.js > HomePage

```
4   const HomePage = () => {
9
10      <Link href={{pathname:"/tab_2", params:{aviation:"Air Canada"}}}>
11          Aller au tab 2 avec passage d'info</Link>
12
```

index.js U

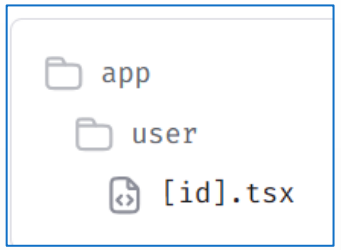
app > (tabs) > tab_2 > index.js > index

```
1   // app/(tabs)/tab_2/index.js
2   import { Text,Text } from 'react-native'
3   import { useLocalSearchParams } from 'expo-router';
4
5   const index = () => {
6       const {aviation="Air"} = useLocalSearchParams();
7       return (
8           <View>
9               <Text>Tab 2: Avion</Text>
10               <Text>Voyagez an toute sécurité avec {aviation}</Text>
11          </View>
12      )
13  }
```

E- Routes dynamiques et passage d'info

Les routes dynamiques et la possibilité de passer un objet params.

Considérez la structure de fichier suivante; chacun de ces liens mènera à la même page :




```
//app/index.tsx
import { Link, router } from 'expo-router';
import { View, Pressable } from 'react-native’;

export default function Page() {
  return (
    <View>
      <Link href="/user/jean"> 1-Afficher utilisateur jean </Link>

      <Link
        href={{ pathname: '/user/[id]',params: { id: 'jean' }}}>
        2-Afficher utilisateur jean
      </Link>

      <Pressable onPress={() => router.navigate({pathname:'/user',
                                 params:{ id: 'jean' } })}>
        <Text>3-Afficher utilisateur jean</Text>
      </Pressable>

    </View>
  );
}
```

Navigating between pages - Expo Documentation